

# Laboratorio 1: filtrado de un ECG

**Autor**

P. E. Caicedo-Rodríguez

**Fecha de publicación**

21 de septiembre de 2026

## 1 Objetivos

1. Identificar en el tiempo y en la frecuencia tres componentes no deseadas de un ECG.
2. Diseñar un pasabanda y un notch con `scipy.signal`.
3. Comprobar que el filtrado de fase cero no desplaza la onda R.

## 2 Materiales

- Python con NumPy, SciPy y Matplotlib.
- `datos_ejemplo.py`: la función `ecg_sintetico` devuelve el ECG limpio y el contaminado.

# 3 Procedimiento

## 3.1 Mirar antes de filtrar

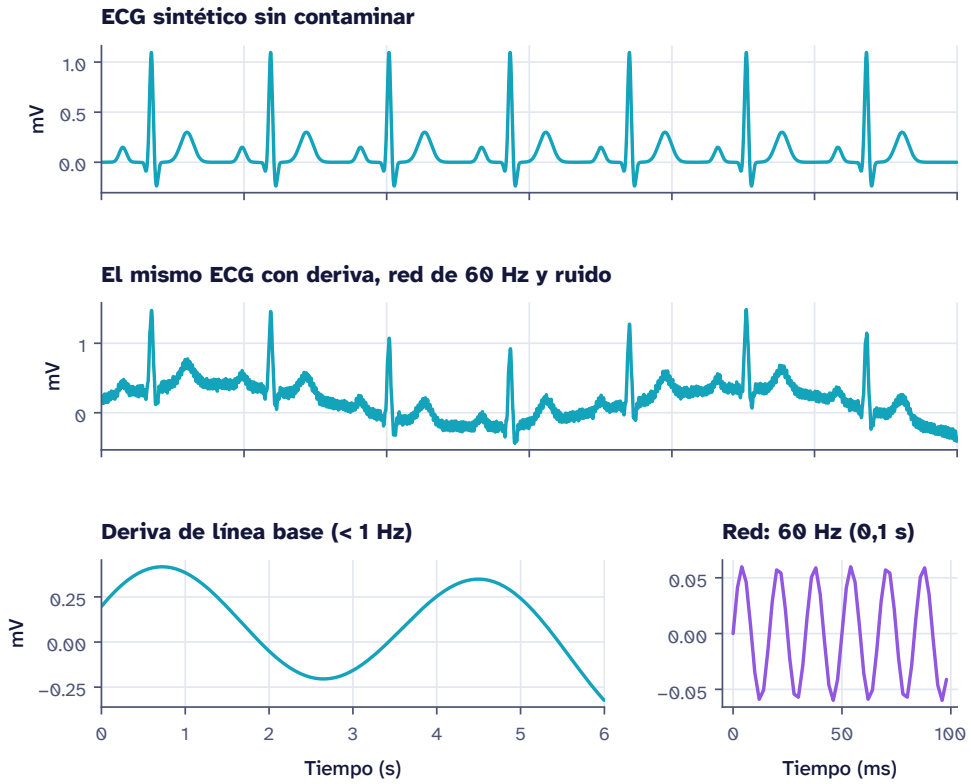



Figura 1: El ECG sintético, contaminado, y dos de sus componentes no deseadas.

```
from datos_ejemplo import ecg_sintetico
t, limpio, sucio = ecg_sintetico(fs=500, duracion=10)
```

## 3.2 Diseñar los filtros

- Pasabanda Butterworth de orden 4 entre 0,5 y 40 Hz.
- Notch de 60 Hz con `signal.iirnotch(60, Q=30, fs=500)`.

### 3.3 Comparar `lfilter` y `filtfilt`

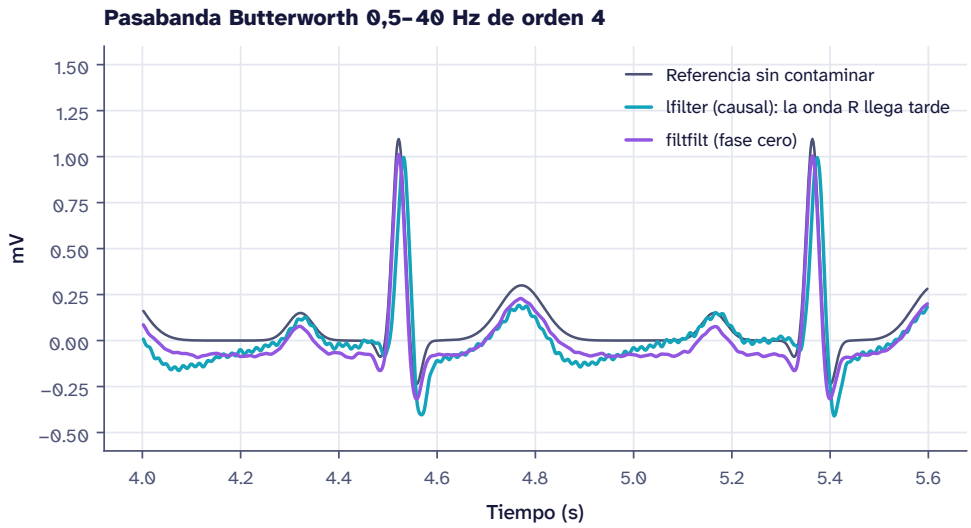



Figura 2: Un latido filtrado con los dos métodos.

Mide el retraso de la onda R en cada caso (por ejemplo con `np.argmax` en una ventana alrededor del latido).

## 4 Preguntas orientadoras

1. ¿Qué componente se ve mejor en el tiempo y cuál en la frecuencia?
2. ¿Por qué un pasa altos de 0,5 Hz no deforma el complejo QRS pero sí podría deformar el segmento ST?
3. ¿Qué pasaría con el notch si la frecuencia de muestreo fuera de 100 Hz?

## 5 Entregables

- Script reproducible con las figuras en el tiempo y en la frecuencia.
- Tabla con el retraso de la onda R para cada método.
- Respuestas a las preguntas orientadoras.

## 6 Rúbrica

Rúbrica del laboratorio 1.

| Criterio | Peso | Logrado |
|---|---|---|
| Identificación de componentes | 25 % | Cada componente se reconoce en el tiempo y en la frecuencia |
| Diseño de filtros | 30 % | Orden, cortes y tipo justificados |
| Fase | 25 % | El retraso se mide y se explica |
| Reproducibilidad | 20 % | El código corre de principio a fin |